

Current State Audit: Amazon Quick Desktop App

1. What Exists Today

1.1 Product Overview

Amazon Quick Desktop App launched April 28, 2026 in preview for macOS and Windows. It extends the web-based Amazon Quick (formerly Quick Suite) into a native desktop experience that runs persistently in the system tray/menu bar. The product positions itself as an always-on AI agent that works across your local filesystem, browser, and connected SaaS applications.

1.2 Core Experience Walkthrough

Persistent Chat Interface. The primary interaction surface is a chat window invoked from the system tray (macOS) or taskbar (Windows). Users type natural-language requests and the agent responds with text, generated content, or by taking actions across applications. The chat persists across sessions, maintaining conversation history and context from prior interactions.

What works: The chat paradigm is familiar and low-friction for first use. Users can immediately ask questions without learning a new interface vocabulary. The persistent nature means users do not lose context between sessions, which is a meaningful advantage over browser-tab-based assistants that reset when closed.

What is broken or missing:

- The chat-first paradigm creates a cold-start problem: new users face a blank text box with no guidance on what the agent can do. There is no structured onboarding wizard, capabilities catalog, or suggested-actions surface.
- Discoverability of capabilities is entirely dependent on the user knowing what to ask for. Power features like background agents, browser automation, and knowledge graph queries are invisible unless the user explicitly invokes them.
- The chat history grows linearly with no threading, folding, or topic-based organization, making it difficult to find prior outputs or revisit earlier work.

Local File Access. The desktop app can read and work with files on the user's computer without uploading them to a cloud service. Users can reference files by path or drag-and-drop them into the chat.

What works: This is a genuine differentiator versus web-based competitors. Processing local spreadsheets, PDFs, and documents without upload latency or size limits removes a significant friction point in existing AI assistant workflows.

What is broken or missing:

- File permission scoping is coarse: the user grants broad filesystem access rather than per-directory or per-session permissions, which raises trust concerns for enterprise security teams.
- No visual file browser or recent-files panel exists within the app itself. Users must know the file path or navigate to Finder/Explorer to drag-and-drop.
- Output files (generated presentations, dashboards, images) lack a clear "project" or output directory concept. Generated artifacts scatter based on implicit defaults.

Browser Automation. Quick Desktop can launch and control Chrome to browse the web, fill forms, extract data, and perform multi-step web workflows on the user's behalf.

What works: Browser automation transforms Quick from a question-answering chatbot into an action-taking agent. The ability to fill forms, extract structured data from web pages, and execute multi-step browser workflows is a category-defining capability that competitors (Copilot, Gemini) do not yet offer at the desktop level.

What is broken or missing:

- Browser automation is Chrome-only in the preview. Safari, Edge, Firefox, and Arc users cannot use this capability.
- The user has limited visibility into what the agent is doing during browser automation. There is no step-by-step progress indicator, no ability to pause/resume, and no confirmation dialog before destructive actions (form submissions, purchases).
- Error recovery during multi-step browser workflows is fragile. If a page layout changes or a CAPTCHA appears, the agent fails silently or produces cryptic error messages.

Proactive Notifications. The app surfaces OS-level notifications for action items, calendar conflicts, unread messages, and other events it deems important.

What works: Proactive behavior is the key differentiator of the desktop form factor versus web. The ability to monitor apps and surface what needs attention without the user opening the app is a strong value proposition for busy professionals.

What is broken or missing:

- Notification volume tuning is absent or rudimentary. Early users report either too many notifications (noisy) or too few (appears inactive). There is no per-source or per-urgency filter.
- Notifications lack deep-link behavior. Clicking a notification opens the Quick chat window but does not navigate to the relevant context (the specific email, calendar event, or Slack thread).
- No "quiet hours" or focus-mode integration with macOS Focus or Windows Focus Assist.

Personal Knowledge Graph. Quick learns your people, projects, and preferences over time, building a graph of relationships that informs future responses.

What works: The promise of a personal knowledge graph that improves over time is compelling and differentiating. It moves the product from stateless chat to a persistent, personalized assistant.

What is broken or missing:

- The knowledge graph is opaque. Users cannot see what Quick has learned about them, correct inaccuracies, or delete specific learned associations.
- There is no explicit mechanism for users to teach the agent (e.g., "When I say 'the board deck', I mean Q2-board-review.pptx on my Desktop").
- Privacy controls for the knowledge graph are unclear, particularly for enterprise deployments where data boundaries between personal and corporate are critical.

Cross-App Integrations. Quick connects to Slack, Teams, Google Workspace, Salesforce, M365, Zoom, Airtable, Dropbox, Jira, and others.

What works: The breadth of integrations is a significant strength. The integration marketplace covers the most common enterprise SaaS tools, and the ability to query across multiple tools from a single chat interface is genuinely useful.

What is broken or missing:

- Integration setup is per-user and manual. Enterprise IT cannot pre-configure integrations for their organization.
- Integration status is not visible at a glance. Users cannot easily see which integrations are connected, which tokens have expired, or which services are experiencing errors.

- Cross-app workflows (e.g., "find the Jira ticket linked to this Slack thread and update it") are limited. Most integrations operate as isolated data sources rather than as a connected graph.

Content Creation. Users can generate presentations, dashboards, intelligent apps, and images directly from chat.

What works: The ability to produce structured output artifacts (not just text responses) from natural-language prompts is a strong differentiator. The generated presentations and dashboards are reportedly polished enough for direct use.

What is broken or missing:

- Generated content lacks iterative refinement. Users can regenerate but cannot say "make the third slide more visual" or "change the color scheme" — they must re-prompt from scratch.
- No template library or style system. Every generation starts from zero rather than from org-approved templates.
- Export formats are limited. Presentations generate as a proprietary format rather than native .pptx, limiting compatibility with existing enterprise workflows.

1.3 Platform-Specific Observations

macOS: The app runs as a menu-bar item with a popover chat window. It follows standard macOS conventions for tray apps but does not integrate with Spotlight, Shortcuts, or the Share Sheet.

Windows: The app runs in the system tray. It does not integrate with Windows Copilot, PowerToys, or the Windows Share dialog.

2. User Pain Map

#	Pain Point	Severity (1-5)	Frequency	Evidence Source
1	Cold-start discoverability gap: blank chat with no guidance on agent capabilities, leading to underutilization of power features (browser automation, background agents, knowledge graph)	4	Daily (every session start)	Product walkthrough; absence of onboarding flow, capabilities catalog, or suggested actions
2	Notification overload with no tuning: users cannot control notification volume by source, urgency, or schedule, resulting in either noise fatigue or perceived inactivity	4	Daily	Product feature inspection; no notification filter, quiet-hours, or OS focus-mode integration observed
3	Opaque knowledge graph: users cannot inspect, correct, or delete what the agent has learned about them, eroding trust and limiting personalization accuracy	4	Weekly	Product walkthrough; no knowledge graph viewer, correction mechanism, or explicit teaching interface
4	Browser automation limited to Chrome with no progress visibility or error recovery	3	Weekly	Feature inspection; Chrome-only limitation, no step-by-step progress UI, silent failures on CAPTCHAs

#	Pain Point	Severity (1-5)	Frequency	Evidence Source
5	Integration status invisible: no dashboard showing connection health, expired tokens, or per-integration error rates	3	Weekly	Product walkthrough; no integration health panel observed
6	Generated content not iteratively refinable: users must re-prompt from scratch rather than editing specific elements of generated presentations, dashboards, or apps	3	Weekly	Feature inspection; no partial-edit or "change slide 3" interaction pattern
7	Chat history grows linearly with no threading, search, or topic organization	2	Daily	Product walkthrough; single-stream chat with no folding or categorization
8	File permission model too coarse for enterprise security requirements	3	Monthly (at setup)	Feature inspection; broad filesystem access grant without per-directory scoping

3. Adjacent Product Inventory

Adjacent Feature	Relationship	Integration Requirement
Amazon Quick Web (browser-based)	Parent product; desktop app extends the web experience	Must maintain conversation continuity and knowledge graph sync between web and desktop. Users expect to start a task on web and continue on desktop (and vice versa).
Amazon Q Developer / Kiro CLI	Developer-facing AI tooling from Amazon	Desktop app advertises Kiro CLI and Claude Code connectivity. Must not conflict with developer workflows; clear separation of "productivity agent" vs. "coding agent" personas.
Microsoft 365 Extensions (Outlook, Word, PowerPoint, Excel)	Preview feature shipping alongside desktop app	Desktop notifications and chat must be aware of actions taken inside M365 extensions. Avoid duplicate notifications or conflicting suggestions between the desktop app and the in-app extensions.
AWS Services Ecosystem	Amazon's cloud infrastructure and enterprise platform	Enterprise customers expect Quick Desktop to connect to AWS accounts, CloudWatch dashboards, and service health. Single sign-on via AWS IAM Identity Center is a likely expectation.
Third-Party SaaS Integrations (Slack, Teams, Google Workspace, Salesforce, Jira, etc.)	Core feature dependency	Each integration requires maintained OAuth tokens, error handling, and rate-limit awareness. New integrations must follow a consistent connector architecture.

4. Current Metrics Baseline

Usage Data: Not publicly available. The product launched in preview on April 28, 2026. Amazon has disclosed enterprise adopters (3M, GoDaddy, AstraZeneca, BMW, Mondelez, NFL, Southwest Airlines, New York Life) but has not published DAU, WAU, or feature-adoption metrics. The free tier suggests Amazon is prioritizing user acquisition over monetization in the preview phase.

Performance Data: Not publicly available. No published latency benchmarks, error rates, or uptime SLAs for the desktop app. The preview label implies performance may vary and no SLA is guaranteed.

Sentiment Data: Early sentiment is cautiously positive based on launch coverage. The product is being recognized for its differentiated desktop agent capabilities (proactive notifications, browser automation, local file access). Concerns center on privacy (knowledge graph data handling), trust (agent taking actions on behalf of users), and the preview-quality limitation of some features.

Baseline metrics not fully available. Researcher should seek: enterprise adoption numbers, feature-adoption rates (which capabilities are most/least used), notification engagement rates (click-through, dismiss, mute), integration connection rates (which SaaS tools are connected most frequently), and churn/retention data for free-tier vs. paid users.

5. Competitor UX Pattern Inventory

No competitor screenshots or demo recordings were provided. The following inventory is based on publicly known interaction patterns of the primary competitors, compiled to feed the Designer and Prototype Builder stages.

#	Pattern	Competitor	Category	UX Detail
1	Sidebar panel embedded in host app	Microsoft Copilot	Navigation	Copilot appears as a right-side panel within Word, Excel, Outlook, and Teams. The user never leaves their host application. Panel is resizable and collapsible. Navigation is flat: one chat thread per document context.
2	@mention invocation in document	Google Gemini	Workflow	Users type @Gemini inside Google Docs, Sheets, or Slides to invoke the assistant inline. The assistant responds in-context (e.g., inserting a generated paragraph at the cursor position). No separate window or panel switch required.
3	System-tray persistent agent with popover	ChatGPT Desktop (OpenAI)	Navigation	ChatGPT Desktop runs as a macOS menu-bar item. Invoked via global keyboard shortcut (Option+Space). Chat window appears as a floating popover. Supports screenshot capture for visual context. Conversation history is accessible via sidebar within the popover.

#	Pattern	Competitor	Category	UX Detail
4	Conversation threading with branching	ChatGPT Desktop (OpenAI)	Data Management	Conversations are organized in a left sidebar with titles. Users can branch conversations, rename them, and search history. Shared links allow exporting a conversation for collaboration.
5	Artifact panel for generated output	Claude Desktop (Anthropic)	Workflow	Claude Desktop renders generated code, documents, and visualizations in a dedicated "artifact" panel alongside the chat. Users can iterate on artifacts ("make the chart blue") without re-prompting from scratch. Artifacts are versioned and downloadable.
6	Deep document co-editing	Microsoft Copilot	Workflow	In Word, Copilot can rewrite selected paragraphs, change tone, summarize sections, and generate content at the cursor. The user reviews a diff-like preview before accepting changes. This is not chat-only — it is embedded in the document editing flow.
7	Integration marketplace with status badges	Microsoft Copilot (M365 admin)	Integration	M365 admin center provides a connector catalog with per-integration status (connected, error, disabled), last-sync timestamps, and admin-level configuration. Enterprise IT can pre-deploy integrations for users.
8	Guided multi-step workflow (wizard pattern)	Google Gemini in Workspace	Workflow	Gemini's "Help me organize" feature in Sheets walks the user through a multi-step process: select data range, choose analysis type, review output, apply. Each step has clear progress indication and back/forward navigation.

#	Pattern	Competitor	Category	UX Detail
9	Proactive suggestion cards	Microsoft Copilot in Outlook	Product Narrative	Copilot in Outlook surfaces suggestion cards above the inbox: "You have 3 unresolved action items from yesterday's meeting" with one-click actions (reply, schedule, delegate). Cards are dismissible and learn from interaction.
10	Global search across connected sources	ChatGPT with memory + integrations	Data Management	ChatGPT's memory feature combined with integrations allows users to search across conversation history and connected data sources from a single search bar. Results show source attribution (which conversation, which integration).

Competitor UX patterns not based on direct artifact review. Researcher should capture interaction patterns (not just capabilities) during competitive analysis, with screenshots where possible.

6. Grounding Constraints for Researcher

- **Must preserve web-to-desktop continuity.** Amazon Quick already exists as a web product with an established user base. Any desktop-specific feature must sync state (conversations, knowledge graph, preferences) with the web experience. The Researcher must not propose desktop-only features that fragment the user's context.
- **Cannot break existing enterprise integration architecture.** The product already connects to 10+ SaaS platforms (Slack, Teams, Google Workspace, Salesforce, M365, Zoom, Airtable, Dropbox, Jira, etc.) via an established connector framework. New integration proposals must follow the existing connector pattern, not introduce a parallel system.
- **Must respect preview-phase limitations.** The product launched less than a month ago in preview. Proposals should account for the fact that core capabilities (browser automation, knowledge graph, background agents) are still maturing. Solutions that depend on these capabilities being production-grade will fail.
- **Must address the trust/transparency gap.** Users are being asked to let an AI agent run in the background, read their files, control their browser, and learn their preferences. Any feature proposal must include a trust/transparency mechanism (audit log, permission dashboard, knowledge graph viewer). The Researcher should treat trust as a first-class product requirement, not a compliance afterthought.
- **Enterprise deployment model is non-negotiable.** The product has enterprise-tier pricing and Fortune 500 adopters. Features must work within enterprise constraints: SSO via AWS IAM Identity Center, admin-managed integrations, data residency controls, and audit logging. Consumer-only features that cannot scale to enterprise governance will not ship.
- **Must coexist with Microsoft Copilot and Google Gemini.** The emerging enterprise pattern is running Quick alongside Copilot/Gemini (different use cases, not replacement). The Researcher should investigate this coexistence pattern and identify where Quick should differentiate (workflow agent, cross-app orchestration) vs. where it should yield (in-app document editing to Copilot, Workspace-native tasks to Gemini).

- **Pricing pressure from established competitors.** At \$20/user/month for Plus, Quick is priced below Copilot (~\$30/user/month) but must justify the cost for organizations already paying for Copilot or Gemini. The Researcher should investigate buyer willingness to pay for a second AI assistant and what capabilities justify the incremental spend.

7. Prototype Surface Requirements

Minimum Page Count

Competing desktop AI assistants (ChatGPT Desktop, Claude Desktop, Copilot) operate with 3-5 primary surfaces: (1) main chat, (2) conversation history/sidebar, (3) settings/integrations, (4) generated output/artifacts panel, and (5) notification/activity feed. Given that Amazon Quick differentiates on cross-app integration and proactive behavior, the prototype should cover a minimum of **6 primary surfaces**:

1. **Main chat interface** with context-aware suggestions
2. **Conversation history** with threading, search, and topic organization
3. **Integration dashboard** showing connected services, health status, and setup flow
4. **Knowledge graph viewer** for inspecting and correcting learned associations
5. **Notification center** with filtering, prioritization, and deep-linking
6. **Generated output gallery** for presentations, dashboards, and artifacts with iterative refinement

Mandatory Interaction Patterns

Based on the competitor UX inventory and pain map, the following interaction patterns are table-stakes for this product category:

- **Global keyboard shortcut invocation** (pattern from ChatGPT Desktop): the agent must be summoned instantly from any context without switching apps
- **Artifact/output panel with iterative refinement** (pattern from Claude Desktop): generated content must be editable in-place, not re-prompted from scratch
- **Integration connector setup with status badges** (pattern from Copilot admin): users and admins must see at-a-glance integration health
- **Proactive suggestion cards with one-click actions** (pattern from Copilot in Outlook): notifications must be actionable, not just informational
- **Multi-step guided workflow** (pattern from Gemini in Workspace): complex tasks like browser automation and cross-app workflows need step-by-step progress indication with pause/resume

Demo Narrative Requirement

A 5-minute demo of this product should walk through the following narrative:

"Morning briefing to cross-app action" — The user opens Quick via keyboard shortcut and receives a proactive morning briefing summarizing overnight emails, Slack messages, and calendar conflicts. The user asks Quick to draft a response to a specific email, review a local spreadsheet for a meeting, and create a presentation from the spreadsheet data. Quick generates the presentation in the artifact panel, the user refines it ("make the executive summary slide more concise"), and Quick exports it. Finally, the user checks the integration dashboard to verify all connected services are healthy, and reviews the knowledge graph to see that Quick correctly linked the meeting, the spreadsheet, and the presentation as part of the same project.

This narrative touches all six primary surfaces, exercises four of the five mandatory interaction patterns, and demonstrates the product's core differentiators: proactive behavior, local file access, cross-app awareness, iterative content creation, and the personal knowledge graph.